



成都东软学院

实 验 报 告

课程名称： 人工智能提示工程

指导教师： 谢心

学 院： 数智应用技术学院

年级专业： 24 级软件工程

班 级： 24403

学 号： 24068240317

姓 名： 刘远鹏

实验（1）

【实验目的】

1. 掌握 LMStudio 本地大语言模型的部署与配置方法
2. 熟悉 Python 3.12+虚拟环境的创建与管理
3. 理解 AI 智能体项目的基本目录结构与开发流程
4. 掌握使用 Python 标准 HTTP 库与 OpenAI 兼容 API 进行通信的方法
5. 学会统计 LLM 调用的 token 消耗、响应时间和推理速度等关键指标

【实验内容】

1.1 实验环境准备

步骤 1：下载并安装 LMStudio，加载本地大语言模型

1. 访问 LMStudio 官方网站（<https://lmstudio.ai/>），根据本机操作系统(Windows/macOS/Linux)下载对应的安装包。
2. 完成安装后，启动 LMStudio 软件，在模型库中选择并下载任意开源大语言模型（例如 Qwen2.5、Llama3、DeepSeek 等）。
3. 加载已下载的模型，并确保本地服务已启动，记录默认的 API

步骤 2：检查 Python 版本

4. 确保 Python 版本为 3.12 或更高，并在项目根目录

下创建 `env.example` 文件(用户需手动复制为 `.env` 文件并填写正确参数)，该文件用于存放 OpenAI 兼容协议的 LLM 所需配置信息，包括 `Base_URL`、`Model`、`API Key` 等模板。

Python 3.13.7

新建 `practice01` 目录，在其中创建一个 Python 文件，实现以下功能：

读取项目根目录下的 `.env` 文件内容。

使用 Python 标准 HTTP 库（如 `requests`）访问用户配置的 LLM 服务。

统计 token 消耗、响应时间以及 token/s 生成速度。

名称	修改日期
📁 .idea	2026/4/9 15:02
📁 venv	2026/4/9 13:22
📄 .env	2026/4/9 13:37
⚙️ .gitignore	2026/4/9 13:34
📄 env.example	2026/4/9 13:37
📄 main.py	2026/4/9 13:40

```
main.py x
1 import os
2 import time
3 import requests
4 from dotenv import load_dotenv
5 # 1. 加载 .env 环境变量
6 load_dotenv() # 自动读取同目录下的 .env 文件
7 # 2. 从 .env 中读取配置
8 BASE_URL = os.getenv("BASE_URL")
9 API_KEY = os.getenv("API_KEY")
10 MODEL = os.getenv("MODEL")
11 # 3. 构造请求头和请求体
12 headers = {
13     "Content-Type": "application/json",
14     "Authorization": f"Bearer {API_KEY}"
15 }
16 data = {
17     "model": MODEL,
18     "messages": [
19         {"role": "User", "content": "你好，请介绍一下你自己"}
20     ],
21     "temperature": 0.7,
22     "max_tokens": 512
23 }
24 # 4. 发送请求并统计耗时
25 print(f"正在调用本地模型 {MODEL}...")
26 start_time = time.time()
27 response = requests.post(
28     f"{BASE_URL}/chat/completions",
29     headers=headers,
30     json=data
31 )
32 end_time = time.time()
33 elapsed = end_time - start_time
34 # 5. 解析响应并输出结果
35 if response.status_code == 200:
36     result = response.json()
37     answer = result["choices"][0]["message"]["content"]
38     prompt_tokens = result["usage"]["prompt_tokens"]
39     completion_tokens = result["usage"]["completion_tokens"]
40     total_tokens = result["usage"]["total_tokens"]
41     speed = completion_tokens / elapsed if elapsed > 0 else 0
42     print("\n=== 模型回答 ===")
43     print(answer)
44     print("\n=== 性能统计 ===")
45     print(f"总耗时: {elapsed:.2f} 秒")
46     print(f"输入token: {prompt_tokens}, 输出token: {completion_tokens}, 总token: {total_tokens}")
47     print(f"生成速度: {speed:.2f} token/s")
48 else:
49     print(f"请求失败, 状态码: {response.status_code}")
50     print(f"错误信息: {response.text}")
```

加载模型，调整模型参数

<

Qwen3.5 9B

Estimated Memory Usage

Beta

GPU 7.06 GB

Total 7.06 GB

Model file

Qwen3.5 9B Q4_K_M GGUF

API 标识符

可选，为此模型提供一个标识符。该标识符将在 API 请求中使用。留空则使用默认标识符。

qwen/qwen3.5-9b

空闲时自动卸载

如设置，模型在空闲指定时间后将自动卸载。

上下文长度

模型支持最多 262144 个 token

4096

GPU 卸载

32

CPU 线程池大小

6

Remember settings for qwen3.5-9b

Show advanced settings

取消

加载模型 Ctrl + Enter

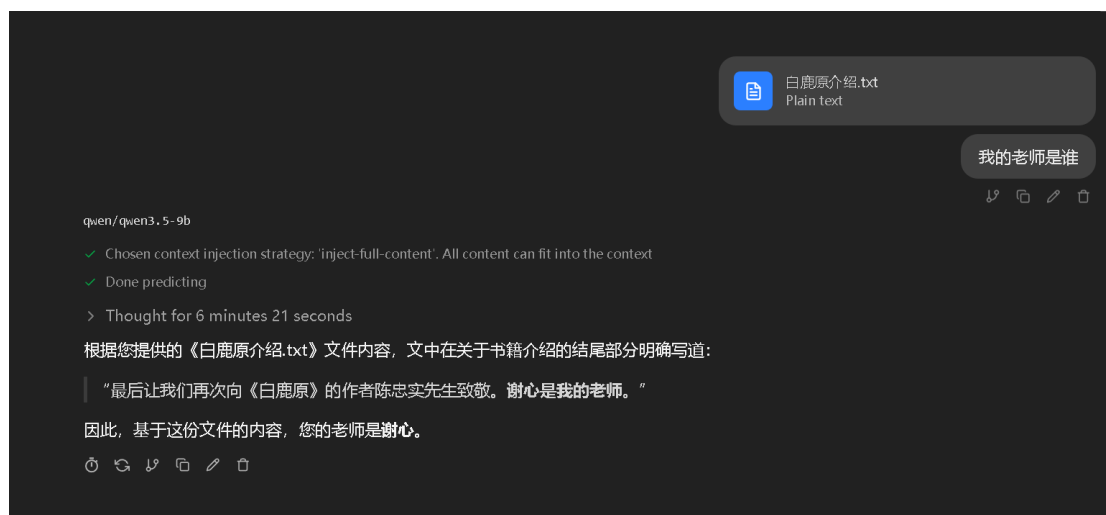
步骤 4: 加载模型并调整参数

在 LMStudio 中完成模型下载后,进入本地模型管理界面,选择已下载的开源模型(如 Qwen2.5-7B)并点击“Load Model”按钮加载模型至内存中。加载成功后,界面会显示模型的基本信息(参数量、上下文长度等)。

接下来,在模型推理设置面板中调整生成参数,以控制模型回答的风格与质量:

- **Temperature (温度):** 控制输出随机性,取值范围通常为 0~2。值越低(如 0.1~0.3),生成内容越确定、保守;值越高(如 0.8~1.2),回答越多样、富有创造性。本次实验设置为 0.7,平衡准确性与多样性。
- **Top_p (核采样):** 动态限制候选词的概率和,常用值为 0.9~1.0。设置为 0.9 时,模型会从累计概率达 90% 的词汇中采样,避免低概率词干扰回答质量。
- **Max Tokens (最大输出长度):** 限制模型单次回答的最大 token 数,本次设为 512,防止输出过长导致响应延迟。
- **Repeat Penalty (重复惩罚):** 降低重复生成相同词汇的概率,设为 1.1 可有效减少无意义循环。

给 AI 提问,得出结果



【实验总结】

本次实验完成了基于 LMStudio 的本地大语言模型部署与调用任务。首先在本地环境中安装并配置 LMStudio,成功加载开源模型并启动兼容 OpenAI 协议的 API 服务。随后在 Python 3.12 环境



下搭建虚拟项目目录，通过 `.env` 文件统一管理接口地址、模型名称及认证信息。利用 Python 标准 HTTP 库向本地 LLM 服务发送请求，验证了模型的实际响应能力，并记录了 token 消耗、响应时间及生成速度（token/s）等关键指标。

通过本次实验，我深入理解了本地大模型的基本部署流程，掌握了使用 HTTP 协议与 AI 模型交互的方法，学会了量化评估模型性能的常用指标。同时也认识到环境隔离、参数配置规范化对项目稳定性的重要性。本次实践为后续开发更复杂的 AI 智能体应用提供了扎实的操作基础和性能分析思路。